In the previous chapters, I often highlighted how cloud native applications should be stateless. We scale them in and out, and if they weren't stateless, we would lose the state every time an instance is shut down. Some state needs to be saved, or the applications would probably be useless. For example, Catalog Service and Order Service are stateless, but they rely on a stateful service (the PostgreSQL database) to permanently store the data about books and orders. Even if the applications are shut down, the data will survive and be available to all the application instances.

Edge Service is not dealing with any business entities it needs to store, but it still needs a stateful service (Redis) to store the state related to the `RequestRateLimiter` filter. When Edge Service is replicated, it's important to keep track of how many requests are left before exceeding the threshold. Using Redis, the rate limiter functionality is guaranteed consistently and safely.

Furthermore, in chapter 11 you'll expand Edge Service to add authentication and authorization. Since it's the entry point to the Polar Bookshop system, it makes sense to authenticate the user there. Data about the authenticated session will have to be saved outside the application for the same reason as the rate limiter information is. If it wasn't, the user might have to authenticate themselves every time a request hits a different Edge Service instance.

The general idea is to keep the applications stateless and use data services for storing the state. As you learned in chapter 5, data services need to guarantee high availability, replication, and durability. In your local environment, you can ignore that aspect, but in production you'll rely on the data services offered by cloud providers, both for PostgreSQL and Redis.

The following section will cover how you can work with Spring Session Data Redis to establish distributed session management.